INFORME ACTUALIZADO - SESIÓN 10

Autenticación JWT, Oracle en contenedores y actividad autónoma sobre expiración y cierre de sesión

ESTUDIANTE	Guillen Mendoza Alexander	**DOCENTE**	David Reyna
ASIGNATURA	Desarrollo de Aplicaciones Móviles	**FECHA DE ACTUALIZACIÓN**	20 de octubre de 2026
PROYECTO	PharmaMobile_Practica + PharmaSoft	**ENTREGA**	GitHub y PDF

1. Resumen ejecutivo

Se integró una sesión JWT de PharmaSoft en la app Kotlin Multiplatform. La solución incluye login, recuperación y cierre de sesión, token cifrado por plataforma, autenticación automática en Ktor, tratamiento distinto de 401 y 403, y secciones REST para productos y clientes. En el servidor se levantaron Oracle Free y Spring Boot mediante Docker Compose, con esquema y datos semilla aplicados por Flyway.

Resultado comprobado: Oracle saludable, cuatro migraciones aplicadas, autenticación y autorización HTTP correctas, CRUD real de un producto temporal (201/200/204), pruebas Maven completas sin fallos y 28 pruebas móviles de host correctas. La ejecución visual de la app contra el backend y la ejecución iOS aún requieren evidencia adicional.

2. Objetivo y contexto

La Guía Práctica N.º 10 solicita integrar el login JWT, adjuntar el token con el plugin Auth de Ktor, protegerlo en Android e iOS, ocultar el token en los registros y distinguir sesión expirada de falta de permisos. Este informe corresponde a la implementación realizada sobre la rama móvil que ya contenía el trabajo de sesiones anteriores. Se conservaron el inventario local demostrativo y el catálogo público de Platzi; Pedidos sigue siendo una demostración, no un CRUD REST.

Los insumos recibidos fueron la guía PDF, una colección Postman de 63 solicitudes de seguridad y un RAR con migraciones V2/V3. El RAR no era una base móvil; su V2 contenía una línea SQL inválida, por lo que se empleó la migración corregida del repositorio PharmaSoft. No se copiaron credenciales ni secretos al código fuente.

3. Arquitectura de la solución

CAPA	RESPONSABILIDAD IMPLEMENTADA
Dominio	Modelos de sesión y errores, contratos de repositorio y casos de uso; sin Ktor, Android ni UIKit.
Datos comunes	DTO, cliente Ktor, Auth Bearer, traducción de errores y repositorios REST de autenticación, productos y clientes.
Plataformas	AlmacenSeguroAndroid con Android Keystore/EncryptedSharedPreferences; AlmacenSeguroIos con Keychain; URL base por expect/actual.
Presentación	ViewModels, LoginScreen, pantallas de PharmaSoft, estados de carga/error, navegación y acciones visibles según permisos.
Servidor	Spring Security/JWT, controladores REST, Oracle Free y migraciones Flyway V1-V4.

4. Funcionalidad móvil implementada

Inicio y recuperación: si no existe token se presenta el login. Con token guardado, la app consulta /auth/me para recuperar identidad y permisos. Un fallo temporal de red permite reintentar sin borrar automáticamente la credencial.

Login: POST /api/v1/auth/login envía usuario y contraseña; el repositorio guarda el JWT en AlmacenSeguro, limpia la caché del proveedor Bearer y descarta la contraseña del estado de interfaz. El DTO de /auth/me se ajustó al campo real empleadoNombre.

Solicitudes protegidas: un cliente Ktor exclusivo de PharmaSoft instala ContentNegotiation, DefaultRequest, Auth Bearer, HttpTimeout y Logging. El token se envía desde la primera solicitud a /api/v1/ excepto al login. No se escribe Authorization manualmente en los composables. El cliente público de Platzi permanece separado.

Seguridad y errores: el logging de depuración usa cabeceras y sanea Authorization como ***; en otras compilaciones está apagado. Un 401 protegido borra la sesión y vuelve al login; un 401 del login significa credenciales inválidas. Un 403 muestra falta de permiso sin cerrar la sesión. El cierre voluntario elimina el token y limpia la caché Bearer.

Permisos y CRUD: CATALOGO_LEER/ESCRIBIR y CLIENTES_LEER/ESCRIBIR determinan qué secciones y botones muestra la interfaz. Las pantallas REST permiten listar, registrar, actualizar y eliminar productos y clientes. La restricción visual no sustituye la autorización del backend.

5. Implementación del servidor y Oracle

Docker Compose ejecuta una imagen oficial de Oracle Free y el backend Spring Boot. Oracle escucha en el puerto local 1522; la API, en el 8080. Los puertos se publican solo en 127.0.0.1. El volumen oracle_data conserva los archivos de la base entre reinicios. Un script idempotente crea PHARMADB al arranque si no existe y aplica sus permisos; se fijó el formato LF del script para evitar errores de Bash en Windows.

El perfil container configura JDBC, Hibernate y Flyway. V1 crea el esquema inicial; V2 agrega empleados, usuarios y roles; V3 siembra usuarios de desarrollo con hashes BCrypt; V4 agrega tres categorías. Se corrigió la configuración CORS requerida por Spring Security. Las contraseñas y el secreto JWT están en un archivo .env ignorado por Git, nunca en el PDF ni en el repositorio.

MIGRACIÓN	RESULTADO COMPROBADO
V1	Esquema inicial del negocio aplicado por Flyway.
V2	Tablas y relaciones de seguridad aplicadas con la versión SQL corregida.
V3	Usuarios y roles de desarrollo disponibles para pruebas JWT.
V4	Tres categorías recuperadas desde la API real.

6. Verificación ejecutada

PRUEBA	EVIDENCIA O RESPUESTA	ESTADO
Oracle / PHARMADB	Contenedor saludable y conexión SQL real	Correcto
Flyway	Cuatro migraciones conservadas tras recrear Oracle	Correcto
GET /api/health	200	Correcto
GET productos sin token	401	Correcto
Login admin y /auth/me	Token emitido; /auth/me devuelve admin	Correcto
GET productos con admin	200	Correcto
GET categorías	Tres registros	Correcto
GET clientes con almacen	403; la sesión sigue autorizada para catálogo	Correcto
Producto temporal	POST 201, GET 200, DELETE 204	Correcto
Pruebas Maven	mvnw.cmd test contra Oracle	Sin fallos
Pruebas y compilación móvil	28 pruebas host; APK debug y Kotlin iOS compilados	Correcto

La prueba del producto usó un registro temporal que se eliminó al terminar. Tras recrear Oracle sin borrar el volumen, siguieron presentes las migraciones, las tres categorías y el login. El flujo real fue comprobado por HTTP desde el equipo anfitrión; no equivale a haberlo recorrido manualmente en la pantalla móvil.

7. Criterios de la guía: estado honesto

CRITERIO	ESTADO
1. Login JWT	Implementado; verificado por HTTP y pruebas móviles
2. Almacén por plataforma	Implementado; compilado, sin prueba visual iOS
3-4. Auth y envío inicial	Implementado; cubierto con MockEngine
5. Log saneado	Implementado y probado; falta captura de Logcat
6. Persistencia tras reapertura	Prueba automatizada; falta ciclo manual de app
7-9. 401, 403 y permisos UI	Probados con mocks; 401/403 reales del servidor comprobados
10. Android e iOS en ejecución	Pendiente de evidencia funcional en ambos

8. Publicación y reproducibilidad

La aplicación se publicó en master y en feature/auth-jwt-guillen del repositorio del estudiante. El backend se publicó en backend/oracle-jwt del mismo repositorio, como rama independiente con sus propios archivos e historial. No debe fusionarse con master como si fuera una característica de la app: es un proyecto distinto. El remoto original del backend, perteneciente al docente, no se modificó.

CONTENIDO	RAMA Y REVISIÓN PUBLICADA
Aplicación	master y feature/auth-jwt-guillen - 5be2535
Servidor	backend/oracle-jwt - a88a22f

Repositorio: github.com/alexgm-A11/PharmaMobile_Practica

Documentación técnica: README.md y docs/S10_AUTENTICACION_JWT.md en master; README.md del backend en backend/oracle-jwt.

9. Limitaciones y siguientes verificaciones

Postman: se analizaron las 63 solicitudes, pero no se ejecutó la colección completa. Sí se verificaron directamente los casos fundamentales de 401, 200 y 403. Falta guardar capturas exigidas por la guía.

Android: el APK compiló y se había instalado/abierto en Pixel 6 sin fallo de arranque. Durante la verificación final no había emulador conectado; faltan el recorrido real de login, CRUD, reapertura con sesión guardada, 403 visible y captura del log con Authorization saneada.

iOS: compiló el código Kotlin compartido para el simulador, pero Windows no ejecuta iosApp ni demuestra el comportamiento del Keychain. La prueba y capturas iOS requieren macOS y Xcode.

Producción: sustituir usuarios semilla y contraseñas de desarrollo, utilizar HTTPS y revisar la estrategia de almacenamiento Android, pues EncryptedSharedPreferences está obsoleta. El tráfico HTTP claro se permite únicamente en la compilación Android debug.

10. Conclusión

La integración JWT y la infraestructura Oracle quedaron implementadas y verificadas en las capas correspondientes. El backend inicia, migra y conserva datos; el acceso protegido y un ciclo real de CRUD responden correctamente. La app dispone del flujo de sesión y CRUD autenticado, con pruebas automatizadas satisfactorias. Para presentar el criterio de funcionamiento en ambas plataformas como completo aún se necesitan las ejecuciones y capturas manuales indicadas.

Fuentes de alcance: Guía Práctica de Laboratorio N.º 10 (13/10/2026), colección Postman de seguridad y migraciones recibidas; código y documentación de las revisiones Git indicadas. Los resultados de prueba corresponden a la verificación local del 13/10/2026.

ANEXO - ACTIVIDAD AUTÓNOMA N.º 10

Validación de expiración, almacenamiento y cierre de sesión

Alcance de este anexo

La actividad se incorporó al informe a partir de la guía y de la revisión del código y de los resultados disponibles. No se modificó el código de la app ni del backend para preparar este anexo. Se distingue entre comportamiento verificado en pruebas o por HTTP, comportamiento identificado por inspección del código y escenarios que todavía requieren ejecución y captura en dispositivo. No se presentan resultados esperados como si fueran observaciones reales.

Producto 1. Bitácora de los seis escenarios de token

La guía exige fecha/hora, plataforma, pasos, código HTTP, mensaje literal, pantalla observada, captura y conclusión propia por escenario. El cuadro resume la evidencia disponible; las capturas y los campos que no fueron registrados quedan expresamente pendientes.

N.º / escenario	Respuesta esperada según guía y backend	Verificación disponible y conclusión
1. Token válido	HTTP 200 en una operación autorizada.	HTTP 200 confirmado al consultar productos con admin y token válido. No se registró mensaje literal (respuesta paginada) ni pantalla/captura Android/iOS. Conclusión: servidor acepta el token válido; falta evidencia en la app.
2. Token expirado	HTTP 401. Mensaje del servidor: El token ha expirado. Inicie sesión nuevamente.	No ejecutado por espera de expiración real. La app trata el 401 protegido como sesión expirada y el mensaje UI previsto es La sesión expiró. Inicia sesión de nuevo.; hay prueba MockEngine del evento 401. No hay respuesta/captura real de expiración.
3. Token alterado	HTTP 401. Mensaje esperado: El token no es válido.	No ejecutado con token alterado. El filtro del backend define ese mensaje; la app convierte cualquier 401 protegido en sesión expirada. Conclusión: falta probar si la pantalla informa de manera adecuada el token alterado.
4. Sin cabecera	HTTP 401. Mensaje esperado: Debe iniciar sesión para acceder a este recurso.	HTTP 401 real confirmado en GET productos sin token. El cuerpo literal y una solicitud posterior al logout no se guardaron. Conclusión: servidor rechaza el acceso anónimo; falta captura del mensaje y UI.
5. Prefijo bearer minúsculo	HTTP 401, tratado como petición anónima.	No ejecutado. El filtro exige el prefijo exacto Bearer y omite otro formato. Sin respuesta HTTP capturada; conclusión basada en revisión del código.
6. Usuario deshabilitado	HTTP 401. Mensaje esperado: El usuario está deshabilitado.	No ejecutado para no cambiar el estado del usuario semilla durante esta actualización documental. El filtro consulta si el usuario está habilitado en cada petición. Falta provocar el caso y registrar respuesta y pantalla.

Fecha/hora y plataforma de las solicitudes directas: 13/10/2026, equipo anfitrión Windows. Las observaciones de token válido y ausencia de token son HTTP al backend; no son capturas de Android/iOS. Los otros cuatro escenarios no se ejecutaron. La guía pide evidencias reales y mensajes del campo message; este anexo no inventa esos datos.

Producto 2. Evidencia de almacenamiento seguro

Comprobación solicitada	Implementación o evidencia disponible	Estado para la entrega
Android: archivo cifrado en shared_prefs	AlmacenSeguroAndroid usa EncryptedSharedPreferences con llave AES256_GCM protegida por MasterKey/Android Keystore; archivo sesion_segura, clave lógica jwt.	Código revisado; no se capturó Device Explorer ni se inspeccionó un dispositivo autenticado. Pendiente captura que demuestre que el valor no está legible.
Authorization saneado en Logcat	Logging sanea la cabecera Authorization. Una prueba MockEngine verifica el saneamiento del registro.	Prueba automatizada disponible; falta captura real de Logcat con Bearer ***.
Sesión tras cerrar y reabrir	El arranque consulta AlmacenSeguro y llama /auth/me; una prueba común restaura un token desde un almacén en memoria simulado.	La prueba simula almacenamiento y no prueba persistencia Android física. Falta iniciar sesión, forzar cierre y reabrir la app con captura.
iOS: servicio y cuenta Keychain	El código usa servicio pe.edu.upeu.pharmamobile.sesion, cuenta jwt y accesibilidad AfterFirstUnlockThisDeviceOnly.	Implementación inspeccionada y source set compilado; no hay Mac/Xcode disponible para verificar lectura/escritura ni captura iOS. Debe declararse esa limitación.

Producto 3. Cierre de sesión completo

Efecto requerido	Estado observado en código/pruebas	Conclusión documental
1. Borrar token del almacén	cerrarSesion llama borrarToken(); prueba común comprueba token nulo en almacén simulado.	Implementado y cubierto en prueba común; falta inspección física del almacén y captura.
2. Olvidar token en caché HTTP	cerrarSesion ejecuta clearToken() del BearerAuthProvider; el login también invalida la caché tras guardar token.	Implementado por código; falta probar la siguiente petición real sin cabecera y registrar el 401.
3. Limpiar estado de interfaz	El ViewModel borra sesión, clave y error. La propiedad usuario se conserva en el estado; el campo de login puede mostrar el usuario anterior.	Hallazgo pendiente: no cumple por completo el requisito de no mostrar el usuario anterior. No se corrigió porque el alcance solicitado fue solo documental.
4. Impedir volver a pantalla autenticada	La app cambia a LoginScreen cuando la sesión es nula; la pantalla autenticada sale de composición. La navegación se mantiene en estado Compose local y no se documentó una prueba del botón Atrás del sistema.	El código descarta el contenido autenticado; falta verificar manualmente que Atrás no restaure la pantalla previa.

Producto 4. Matriz de permisos por rol

El API real comprobó admin en productos y /auth/me, y almacén recibiendo 403 en clientes. El resto de combinaciones se consigna como resultado esperado de la práctica/servidor y oferta derivada de los permisos en el código, no como ejecución real. La app móvil no tiene ventas REST ni una sección de reportes: Pedidos es una pantalla demostrativa filtrada por VENTAS_LEER.

Operación	Rol	HTTP esperado	Interfaz según código	HTTP real capturado
Listar productos	admin	200	Visible	200
Listar productos	vendedor	200	Visible	No ejecutado
Listar productos	almacen	200	Visible	No ejecutado
Registrar producto	admin	201	Visible	201 (temporal)
Registrar producto	vendedor	403	Oculta	No ejecutado
Registrar producto	almacen	201	Visible	No ejecutado
Listar clientes	admin	200	Visible	No ejecutado
Listar clientes	vendedor	200	Visible	No ejecutado
Listar clientes	almacen	403	Oculta	403
Registrar venta	admin	201	Pedidos demo; sin venta REST	No ejecutado
Registrar venta	vendedor	201	Pedidos demo; sin venta REST	No ejecutado
Registrar venta	almacen	403	Sección Pedidos oculta	No ejecutado
Ver reportes	admin	200	Sin pantalla de reportes	No ejecutado
Ver reportes	vendedor	403	Sin pantalla de reportes	No ejecutado
Ver reportes	almacen	403	Sin pantalla de reportes	No ejecutado
Consultar /auth/me	admin	200	Usuario/rol en sesión	200
Consultar /auth/me	vendedor	200	Usuario/rol en sesión	No ejecutado
Consultar /auth/me	almacen	200	Usuario/rol en sesión	No ejecutado

Evaluación frente a la actividad

La base de sesión, el borrado del token, la limpieza de caché Bearer y la visualización por permisos existen en el proyecto. Hay pruebas automatizadas para 401/403 y para ocultar clientes/ventas a almacén. Sin embargo, esta actualización no incluye bitácora experimental completa, capturas por escenario, evidencia física de almacenamiento, recorrido manual de logout, ni una matriz HTTP completa de 18 celdas. Además, se registra que el nombre de usuario permanece en el campo de login tras el logout y que ventas/reportes no tienen integración REST en esta rama.

Estado de la entrega: informe complementado con la especificación y con el estado comprobable del proyecto. Los resultados marcados 'No ejecutado' y las capturas pendientes no deben presentarse como experimentos realizados. Fecha límite indicada por la guía: lunes 19 de octubre de 2026 a las 23:59.

ANEXO - SESIÓN 11: PERSISTENCIA OFFLINE-FIRST

Actualización documental solicitada según la Guía Práctica de Laboratorio N.º 11

1. Alcance y criterio de honestidad

Este anexo actualiza el informe existente a partir de la guía de sesión 11, fechada 20 de octubre de 2026. El encargo actual es actualizar el informe; por ello, no se modificó el código de PharmaMobile ni PharmaSoft. Se revisaron los archivos de la rama móvil feature/auth-jwt-guillen. La guía describe el objetivo y los pasos para una implementación offline-first, pero sus instrucciones no constituyen evidencia de que esos pasos ya estén implementados en este proyecto.

Estado observado en la revisión: persistencia offline-first aún no implementada en la rama inspeccionada. El repositorio de productos que existe es en memoria, el contrato expone listar() como una lectura puntual, y el ViewModel carga manualmente y vuelve a cargar después del registro. No se encontró configuración, esquema ni driver SQLDelight. En consecuencia, no se afirma que la aplicación abra sin PharmaSoft, sincronice pendientes ni cumpla la entrega de sesión 11.

2. Requisitos de la guía y estado actual

REQUISITO DE SESIÓN 11	QUÉ DEBE EXISTIR / CÓMO SE COMPRUEBA	ESTADO DE LA RAMA REVISADA
Esquema SQLDelight	Plugin 2.1.0, PharmaDb y tabla producto con marca pendiente.	No encontrado: shared no aplica plugin SQLDelight ni contiene archivos .sq.
Drivers por plataforma	Creador de driver Android, iOS y driver de memoria/JDBC para pruebas; registro en Koin.	No encontrado: no hay driver de base registrado en las plataformas ni base PharmaDb en dataModule.
Fuente de verdad local	La pantalla lee siempre desde la base local mediante Flow; las respuestas remotas actualizan esa base.	No cumple todavía: ProductoRepository.listar() devuelve List; ProductoRepositorioEnMemoria guarda en una lista mutable.
Crear sin conexión	Guardar localmente como pendiente con id negativo, reflejarlo inmediatamente y no perderlo ante errores de red.	No implementado: registrar() solo agrega al repositorio en memoria; no hay cola ni marca pendiente.
Sincronización	Enviar primero pendientes; luego descargar y reemplazar solo filas sincronizadas, dentro de transacción.	No implementada: no existe sincronizar() ni repositorio offline.
ViewModel	Suscribirse una vez a observar().collect; sincronizar por separado y mostrar aviso manteniendo datos guardados.	No cumple todavía: carga puntual y recarga manual después de guardar.
Pruebas commonTest	Al menos tres pruebas; cada una con base en memoria. Cubrir camino online y offline.	No encontradas para persistencia o sincronización offline. Las pruebas comunes actuales cubren otras funciones.
Evidencia de aceptación	Android e iOS sin red, alta en modo avión, reconexión, producto en Swagger con id positivo y salida de pruebas.	Pendiente: no se aportó evidencia de este ciclo offline ni capturas Android/iOS para la sesión 11.

3. Implementación que la práctica requiere

El trabajo de código pendiente debe recorrer las capas sin hacer que UI dependa de SQLDelight o Ktor: (1) declarar PharmaDb y Producto.sq en commonMain; (2) definir CreadorDeDriver y crear drivers nativos Android/iOS, registrándolos en Koin; (3) encapsular consultas y conversión a Producto en ProductoLocal; (4) cambiar el puerto ProductoRepository para exponer observar(): Flow>, registrar() y sincronizar(); (5) implementar ProductoRepositorioOffline que guarda primero local, intenta subir pendientes y solo después descarga; y (6) suscribir ProductoViewModel al Flow y mantener la sincronización como acción separada. La lista UI debe derivar de la base local, no de una respuesta HTTP.

Para evitar pérdida de datos, la guía propone ids locales negativos, conservar filas pendientes al reemplazar datos del servidor y usar una transacción para borrar/reinsertar la colección sincronizada. Un error ErrorApi.SinConexion no debe cancelar el guardado local. La sincronización correcta es: subir pendientes -> obtener el catálogo remoto -> actualizar local. Invertir ese orden puede borrar el único ejemplar local de un producto creado offline.

4. Plan de verificación y evidencia todavía necesaria

ETAPA	PROCEDIMIENTO SOLICITADO	EVIDENCIA QUE DEBE ADJUNTARSE
Pruebas unitarias	Ejecutar al menos tres pruebas en commonTest con una base SQLDelight en memoria independiente por prueba; cubrir visualización local y sincronización/fracaso de red.	Salida real del comando Gradle y nombres/resultados de las pruebas. No sustituir por el código de la prueba.
Android offline	Sincronizar conectado, cerrar app, activar modo avión, abrir y comprobar lista/aviso; registrar un producto sin red.	Captura legible de la lista local y del producto recién creado mientras el emulador está en modo avión.
Reconexión	Desactivar modo avión y ordenar sincronización; después comprobar que el producto existe en Swagger con id positivo asignado por el servidor.	Capturas del estado de red/sincronización y del recurso en Swagger con id real. Esto acredita la subida, no solo la lectura local.
iOS offline	Repetir apertura, lectura local y registro sin conexión en dispositivo/simulador iOS; volver a sincronizar.	Capturas de iOS y verificación de que el driver nativo se crea correctamente. No se incluyó evidencia de ejecución iOS en esta actualización.
Documentación	Actualizar README con estructura de paquetes, decisiones y apartado Persistencia offline-first; entregar rama feature/offline-first- y commits distribuidos.	Enlace/revisión de la rama, listado de commits y sección README actualizada. Esta actualización no creó rama ni commit.

La guía es explícita: ver el producto en la pantalla no demuestra que haya sido enviado. La verificación del ciclo solo se completa cuando, tras reconectar, el servidor muestra el producto con su identificador positivo. De forma similar, compilar el proyecto no demuestra que iOS esté leyendo la base local; se requieren ejecución y capturas en ambas plataformas.

5. Conclusión actualizada

La rama examinada conserva la sesión 10 (JWT y backend) que se detalla en el cuerpo previo del informe, pero todavía no satisface el criterio nuevo de persistencia local offline-first de la sesión 11. La capa de productos sigue siendo demostrativa/en memoria y no reactiva a través de un Flow de base de datos. Este PDF incorpora lo que pide la guía, el contraste con el estado observado y el plan de aceptación; no es un certificado de implementación ni de pruebas offline. Para declarar completa la sesión se deben realizar los cambios de código, correr al menos tres pruebas commonTest y adjuntar las evidencias de Android, iOS y Swagger descritas arriba.

Fuente para este anexo: Guía Práctica de Laboratorio N.º 11, “Persistencia local con SQLDelight y estrategia offline-first”, 20/10/2026; revisión de shared/build.gradle.kts, ProductoRepository, ProductoRepositorioEnMemoria y ProductoViewModel de la rama feature/auth-jwt-guillen. Estado documental revisado el 20/10/2026.